SEDust: User Manual

Abstract

This manual describes the model-agnostic dust library under `SEDust/sed/`. It computes the infrared emission spectrum *and* the transport optics (extinction, scattering, asymmetry) of a dust mixture for an arbitrary local radiation field, and is intended to be linked into a Fortran 3D radiative-transfer (RT) code. It handles several dust models as peers — the HD23 *astrodust*+PAH model, the Draine & Li (2007) silicate+carbonaceous model, the Zubko, Dwek & Arendt (2004) BARE-GR-S model, and arbitrary file-defined models — through one object type, one emission call, and one extinction call. The numerically validated stochastic-heating solver is reused unchanged; the library is an additive layer on top of it.

This version is the **polarized** branch (v1.20). It is a *superset* of the scalar branch (v1.00): every scalar capability documented here is present there and computes the same numbers, and v1.20 adds the polarized cross sections, the direction-dependent extinction matrix and the aligned-grain scattering matrix (§6.). The one non-additive difference is that the `status` codes of `sed_init` / `build_astrodust` are numbered differently, because codes 3–5 are taken by the polarized inputs: what is `3` and `4` in v1.00 is `6` and `7` here (§4.5). Code that only tests `status /= 0` is unaffected.

Contents

1. Overview **2**

2. Building the library **3**

3. Models and builders **3**

4. Computing emission **5**

4.1 Single-cell exact solve 5
4.2 Solver choice (`m%stoch_method`) 5
4.3 Equilibrium-temperature options 5
4.4 Precomputed table + interpolation 6
4.5 Optional error status 6

5. Computing extinction **7**

6. Polarized dust optics **11**

6.1 Polarized emission 11
6.2 Polarized extinction 12
6.3 Changing the alignment efficiency 12
6.4 Where the polarized optics come from 14
6.5 Aligned-grain scattering optics 14
6.6 Division of labor 16
6.7 Minimal example 17
6.8 Limits 17

7. The extreme-ultraviolet extension **18**

7.1 Where the optics come from in the extended band . . . 18

7.2 The shape approximation, and why it is bounded . . . 19

7.3 The polarized optics do not extend with the grid . . . 19

7.4 The single-photon ceiling comes from the field . . . 20

7.5 What else the extension does, and does not, change . . . 22

7.6 How far each model can be checked . . . 22

8. Standalone programs **23**

8.1 `calc_kext.x` — size-integrated transport optics . . . 23

8.2 `main_astrodust.x` — the astrodust production SED . . . 24

8.3 `main_dl07.x` — the DL07 reference SED . . . 24

8.4 `main_zubko.x` — the Zubko/ZDA SED, and the wide-grid regression . . . 24

8.5 `make_enthalpy.x` — enthalpy tables (diagnostic) . . . 25

8.6 Polarized and diagnostic programs . . . 25

8.7 Polarized optics at other wavelengths . . . 25

9. The model object and accessors **26**

10. The generic file loader **27**

11. Linking into a 3D RT code **28**

12. Units and conventions **28**

13. Data files **28**

1. Overview

A *dust model* is represented by a single derived type, `dust_model_t`. You build it once (per RT run), then call `dust_emission` once per RT cell with that cell's local mean intensity:

```
use dust_lib
type(dust_model_t) :: m
real(wp), allocatable :: J_lam(:), lamI_total(:)

call build_astrodust(m, qtab, sizedist, 200, 2.7_wp, 5.0e3_wp) ! once
allocate(J_lam(m%NLAM), lamI_total(m%NLAM))
do icell = 1, ncells
   ! ... assemble J_lam(:) (mean intensity on the grid m%lam) for this cell ...
   call dust_emission(m, J_lam, lamI_total) ! per cell
   ! ... lamI_total(:) is lambda*I_lambda / N_H for the cell ...
```

```
end do
```

Design. The library wraps the existing, FIR-validated solver core (`sed_grain_loop`, `calc_Teq`, `calc_P`) without modifying it. A model is a set of grain *populations* (one per species and charge state), each carrying its own size distribution, absorption cross sections, enthalpy, and Planck-integral tables. `dust_emission` loops the populations through the solver and sums their emission into named output *channels*. The same populations carry the extinction-side optics, so `dust_extinction` (§5.) returns the model's opacity from that one object as well, and emission and extinction both reach the transfer code through calls on one model. Its scalar cross sections are served out of a size-integrated table the builder attached (§5.); the polarized ones are computed on every call, because they follow the host's runtime alignment state.

One active model at a time. The module-global grids (`lam`, `T_first`, ...) hold the *currently built* model's working set. Calling `build_*` makes that model active. This is sufficient for an RT run that uses a single dust model; to switch models, rebuild.

2. Building the library

From `SEDust/sed/`:

```
make libsedust.a # the library archive (link this into your RT code)
```

The compiler is `gfortran`. The default flags are `-O3 -ffree-line-length-none -fopenmp` plus a warning set — deliberately *portable*, so that a `libsedust.a` built on one node runs on any other. A `-march=native` line is present but commented out; enabling it can change the last digits through FMA contraction. `libsedust.a` is a static archive of all library objects; the Fortran `.mod` files are written alongside it in `SEDust/sed/`. Because those `.mod` files share one directory, the `Makefile` carries `.NOTPARALLEL` and builds serially; `make -j` is not supported until the object and module graph lives in a separate directory.

3. Models and builders

Each builder fills a `dust_model_t` and defines the model's output channels (Table 1).

builder	model	channels	notes
`build_astrodust`	HD23 astrodust+PAH	`AD`, `PAH`	N_C=417, D16 graphite
`build_dl07`	Draine & Li 2007	`SIL`, `CARB`	N_C=470, D03 graphite
`build_zubko`	Zubko (ZDA 2004) BARE-GR-S	`PAH`, `GRA`, `SIL`	neutral PAH only
`build_from_files`	file-defined	`CH1`, `CH2`, ...	generic loader

Table 1: Built-in model builders. Each sets the model's own optics/abundance parameters internally.

Signatures (all optional arguments in brackets; every optional is a keyword argument in the order shown):

```
call build_astrodust(m, qtable_path, sizedist_path, NT, T_lo, T_hi &
                  [, status] [, qpol_path] [, qpol_wave_path] [, qpol_aeff_path] &
                  [, scatmat_path] [, load_polarized_optics] &
                  [, lam_min] [, astrodust_index_path] &
                  [, qpol_euv_path] [, qpol_euv_wave_path])
call build_dl07 (m, qtable_path, sizedist_path, sd_index, U, NT, T_lo, T_hi &
                  [, status] [, lam_min])
call build_zubko (m, config_path, data_dir, NT, T_lo, T_hi [, status])
call build_from_files(m, descriptor_path, data_dir, NT, T_lo, T_hi [, status])
```

The polarized arguments (`qpol_*`, `scatmat_path`, `load_polarized_optics`) are described in §6.; the rest are common to both branches.

- `NT`, `T_lo`, `T_hi`: number and range [K] of the internal temperature grid (typical `200, 2.7, 5000`).
- `sd_index` (DL07): WD01 size-distribution index — `7` = MW R_V=3.1, $q_{\rm PAH}$=4.58%; `29/30/31` = LMC2; `32` = SMC.
- `U` (DL07): ISRF intensity scaling (the DL07 reference uses U=1).
- `data_dir` (Zubko / files): directory holding the data files, e.g. `data/zubko/`.
- `status` (optional, all builders): error report; see §4.5. When present, a missing or malformed input file is reported through it and the model is not built, so an RT host can recover.
- `lam_min` (optional, astrodust and DL07): shortest wavelength [μm] the model must cover, for a host that transports ionizing radiation. See §7.. **Omitting it reproduces the previous behavior exactly** — the grid, and every number derived from it, is unchanged.
- `astrodust_index_path` (optional, astrodust only): the dielectric function that supplies the optics below 0.0912 μm. It must be the file `qtable_path` was computed from — same porosity, iron fraction and axial ratio — or the model changes material at the seam. Omitted, it is the default that pairs with the shipped Q table:

  ```
  data/dielectric/index_DH21Ad_P0.20_0.00_1.400 <->
  tmatrix/output/q_astrodust_P0.20_Fe0.00_1.400.dat
  ```

Two routes to the Zubko model. `build_zubko` evaluates the ZDA log-polynomial size-distribution *formula* (coefficients read from `ZDA_BARE_GR_S_Config.dat`) on the optics-table radii; `build_from_files` with `data/zubko/zubko_descriptor.txt` instead interpolates the *tabulated* `SzDist` files onto the same radii. The two shapes agree to $\sim 10^{-5}$, but the distributed tables carry a normalization that sits below $A\,g(a)$ of the config by 0.79% (PAH), 1.13% (graphite) and 0.41% (silicate), and they are sampled at only 24 / 62 / 63 radii, so interpolation near $a_{\rm max}$ adds a little more. Together this moves the size-integrated $C_{\rm ext}$ by up to 2.10% (median 1.02%). **The formula is the default and is faithful to the ZDA definition**; the tabulated route exists to reproduce the benchmark's own size grid.

4. Computing emission

4.1 Single-cell exact solve

```
call dust_emission(m, J_lam, lamI_total [, lamI_chan] [, status])
```

- `J_lam(m%NLAM)`: the local *mean intensity* on the grid `m%lam` [μm]. For a scaled Mathis ISRF use `call J_Mathis(U, m%lam, J_lam)` from module `radfield`.
- `lamI_total(m%NLAM)`: the summed SED, $\lambda I_\lambda / N_\mathrm{H}$ [erg s^{-1} cm^{-2} sr^{-1} H^{-1}] — the HD23 convention.
- `lamI_chan(m%NLAM, m%n_channel)` (optional): the SED of each channel.
- `status` (optional): error report; see §4.5.

4.2 Solver choice (`m%stoch_method`)

The field `m%stoch_method` selects how each grain is heated (Table 2); `dust_emission` honors it.

`m%stoch_method`	method	note
`'heuristic'`	GD look-ahead T-window narrowing	**default**; fastest, serial
`'draine'`	Guhathakurta & Draine (1989)	reference GD
`'qm'`	energy-space transition matrix (thermal-discrete)	most detailed
`'equil'`	single-grain equilibrium (no stochastic)	see §4.3

Table 2: Stochastic-heating solver options. The first three resolve the grain temperature *distribution* (PAH/NIR features); `'equil'` forces equilibrium.

4.3 Equilibrium-temperature options

Two options replace the full stochastic solve when an equilibrium approximation is wanted:

(1) Equilibrium temperature for each grain type and size. Set `m%stoch_method = 'equil'` and call `dust_emission`. Every grain is placed at its own equilibrium temperature T_eq, computed from *that grain's* absorption cross section, and emits $C_\mathrm{abs}\, B_\lambda(T_\mathrm{eq})$. No stochastic excursions are computed.

(2) A single equilibrium temperature for the whole model.

```
call dust_emission_single_teq(m, J_lam, lamI_total [, Teq_out])
```

All dust shares one temperature, found from the type- and size-integrated (dn-weighted) total absorption cross section $C^\mathrm{tot}_\mathrm{abs}(\lambda) = \sum_\mathrm{pop} \sum_a \mathrm{d}n(a) C_\mathrm{abs}(\lambda, a)$. The single T_eq satisfies $\int C^\mathrm{tot}_\mathrm{abs} J \,\mathrm{d}\lambda = \int C^\mathrm{tot}_\mathrm{abs} B(T_\mathrm{eq}) \,\mathrm{d}\lambda$, and the emission is $C^\mathrm{tot}_\mathrm{abs}\, B_\lambda(T_\mathrm{eq})$. The optional `Teq_out` returns that temperature.

Both options conserve energy (the emitted bolometric power equals the absorbed power); they differ from the stochastic solve only in the SED *shape* — a smooth modified blackbody rather than one carrying PAH/NIR features.

4.4 Precomputed table + interpolation

When the field in each cell is a fixed spectral *shape* scaled by an intensity U, precompute a table once and interpolate per cell:

```
type(dust_emis_table_t) :: tab
call dust_build_table(m, J_ref, U_grid, tab [, status]) ! once
call dust_emission_interp(tab, U, lamI_total [, lamI_chan] [, status]) ! per cell
```

`J_ref(m%NLAM)` is the reference field shape (at U=1) and `U_grid(:)` an ascending intensity grid. The table is built from exact solves, so it is exact at the grid points; interpolation is log–log in U. The stored emissivities are floored at 10^{-300} before the log, so a grid node whose true value is 0 comes back as 10^{-300} rather than exactly 0. Both calls take an optional final `status` (§4.5).

Caveat. Stochastic emission is a *nonlinear* functional of the full $J(\lambda)$, not of a scalar. The table is valid only when the cell field is $U \times$ (fixed $J_{\rm ref}$). For cells whose field *shape* departs from $J_{\rm ref}$ (e.g. hardened spectra near hot stars), use the cell-by-cell exact `dust_emission`.

4.5 Optional error status

The builders and the emission/table calls each accept an optional final `status` argument (`integer, intent(out)`). When it is present, an invalid model, argument, or input file is reported through it (`status = 0` on success, nonzero on failure) and the process keeps running, so a 3D RT host can recover; when it is omitted, the same condition prints a message and `stop`s the run, which the CLI drivers rely on. For the builders the nonzero value only names the failing stage: the contract is simply `0` = built, nonzero = build failed.

- `dust_emission`: `1` unknown `stoch_method`; `2` `'qm'` selected but a population's radii are unset.
- `dust_build_table`: `1` `size(J_ref)`$\neq$`m%NLAM`; `2` `size(U_grid)` < 2; `3` `U_grid` not positive and strictly increasing.
- `dust_emission_interp`: `1` `U` ≤ 0; `2` `size(lamI_total)`$\neq$`tab%NLAM`; `3` `lamI_chan` present but not `(tab%NLAM, tab%n_channel)`.
- `build_dl07`: `1` Q-table load failed; `2` size-distribution load failed; `7` `lam_min` is shorter than the D03 dielectric functions ($6.1992 \times 10^{-5}\,\mu$m — the longer-reaching of astrosilicate and graphite), which is refused rather than served with a refractive index frozen at the table boundary. Codes `3`–`6`, `8` and `9` cannot occur on this path: the DL07 model has no polarized optics and needs no separate EUV dielectric function, because its optics are D03 Mie at every wavelength. The numbering is shared with `build_astrodust` on purpose, so a host can decode one table.
- `build_astrodust` (and `sed_init`): `1` Q-table load failed; `2` size-distribution load failed; `3` an explicit `scatmat_path` failed to load; `4` an explicit `qpol_path` could not be read (an implicit default degrades gracefully

instead); 5 `load_polarized_optics=.false.` combined with an explicit `qpol_*/scatmat_path` (a contradiction); 6 the astrodust dielectric function failed to load (raised only when `lam_min` asks for the EUV band); 7 `lam_min` is shorter than that dielectric function's own shortest wavelength ($1.00003 \times 10^{-4}\,\mu$m), refused for the same reason as above; 8 the EUV companion polarized table could not be read while the grid reaches into that band and polarized optics were asked for; 9 the EUV part of the grid runs outside the wavelengths that table covers (0.0124–0.0912 μm). **Codes 6 and 7 are 3 and 4 in the scalar branch (v1.00)**, which has no polarized inputs to occupy 3–5.

- `build_zubko`: 1 config read; 2 fewer than 3 components; 3 a component's optics read; 4 grid inconsistent; 5 a component's calorimetry read failed; 6 an explicitly named `kext_path` failed to load.
- `build_from_files`: 1 descriptor open; 2 too many `pop:` lines; 3 invalid channel; 4 no `pop:` lines; 5 optics read; 6 grid inconsistent; 7 size-distribution read; 8 calorimetry read failed; 9 an explicitly named `kext_path` failed to load.
- **An explicitly named `kext_path` that fails to load** (§5.) is 5 in `build_dl07`, 6 in `build_zubko`, 9 in `build_from_files` — the next free code in each — and 10 in `build_astrodust`, whose polarized codes already occupy 3, 4, 5, 8 and 9. The scalar branch (v1.00) uses the same numbers except that its `build_astrodust` has no polarized codes and so uses 5 there too.
- `dust_extinction`: 1 an output array is not of size `m%NLAM`; 2 no extinction table was loaded for this model; 3 `m%lam` runs outside the table.

5. Computing extinction

`dust_extinction` is the extinction counterpart of `dust_emission`: a transfer code takes its opacity from the same model object, and on the same wavelength grid `m%lam`, as its emission.

```
call dust_extinction(m, Cext, Cabs, Csca [, gbar] [, Cpol_ext] [, Cbir_ext] &
                  [, albedo] [, status])
```

- `Cext`, `Cabs`, `Csca` (`m%NLAM`): extinction, absorption and scattering cross sections per H atom [cm^2/H], with $C_{\rm ext} = C_{\rm abs} + C_{\rm sca}$.
- `gbar(m%NLAM)` (optional): the scattering asymmetry $\langle \cos\theta \rangle$.
- `Cpol_ext(m%NLAM)` (optional): the dichroic (polarized) extinction [cm^2/H]; see §6..
- `Cbir_ext(m%NLAM)` (optional): the birefringent extinction [cm^2/H], the $f_{\rm align}$-weighted size integral of the orientation-resolved table's 4th block. It is 0 when the loaded table carries no birefringence (the release table does not); see §6..
- `albedo(m%NLAM)` (optional): $C_{\rm sca}/C_{\rm ext}$, and 0 where $C_{\rm ext}$ underflows to zero. It is derived here rather than by the caller so that every host gets the same convention at those wavelengths.

- `status` (optional): error report; see §4.5. It is 0 on success, 1 if an output array is not of size `m%NLAM`, 2 if no extinction table was loaded for this model, and 3 if `m%lam` runs outside the table.

Where the numbers come from. The four *scalar* outputs — `Cext`, `Cabs`, `Csca`, `gbar` — are read from a precomputed size-integrated extinction table, one of the `data/kext_*.dat` products of `calc_kext.x` (§8.), attached to the model at build time and interpolated onto `m%lam`. The transport optics of a dust model do not depend on the transport, so there is nothing to gain by repeating the size integral during a transfer run: the table *is* that integral, done once and recorded.
The two *polarized* outputs — `Cpol_ext` and `Cbir_ext` — are **not** tabulated. They are computed from the model's own orientation-resolved optics on every call, because their $f_{\rm align}(a)$ weight is runtime state a transfer code resets cell by cell through `dust_set_alignment` (§6.3), which a table fixed at build time could not follow. This asymmetry — scalar optics from a table, polarized optics from the model — is deliberate.
The first-principles size integral remains available under its own name and with an identical argument list:

```
call size_integrated_extinction(m, Cext, Cabs, Csca [, gbar] [, Cpol_ext] &
                            [, Cbir_ext] [, albedo] [, status])
```

It needs no table and computes all six outputs from the model's optics. It is what `calc_kext.x` calls to write the tables in the first place, and what the in-tree checks (`test_scalar_mode.x`, `test_euv_extension.x`) call when they have to compare one build of a model against another. Each output of it is the plain binned sum over every population of the model, since $\mathrm{d}n(a)$ already carries the bin width:

$$C_{\rm abs}(\lambda) = \sum_{\rm pop}\sum_{a} \mathrm{d}n(a)\,C_{\rm abs}(\lambda,a), \qquad C_{\rm sca}(\lambda) = \sum_{\rm pop}\sum_{a} \mathrm{d}n(a)\,C_{\rm sca}(\lambda,a),$$
$$\langle\cos\theta\rangle = \frac{\sum \mathrm{d}n\,C_{\rm sca}\,g}{\sum \mathrm{d}n\,C_{\rm sca}}, \qquad C_{\rm pol,ext}(\lambda) = \sum_{\rm pop}\sum_{a} \mathrm{d}n(a)\,C_{\rm pol,ext}(\lambda,a)\,f_{\rm align}(a). \tag{1}$$

A population that carries no scattering or polarized optics contributes zero to those terms. In the astrodust model the PAHs are exactly that case: they enter through absorption only, and the astrodust grains carry all the scattering and the asymmetry.

Dust mass per H, and mass opacity. Both routines return cross sections *per H nucleon*. A host that carries a dust mass density instead of a hydrogen column converts them with one number, which the model itself reports:

```
Mdust_H = dust_mass_per_H(m) ! [g/H]
kappa = Cabs / Mdust_H ! [cm^2 per gram of dust]
```

It is the model's own size distribution weighted by the solid density of each population's material,

$$\frac{M_{\rm dust}}{N_{\rm H}} = \sum_{\rm pop} \rho_{\rm bulk} \sum_{a} \tfrac{4}{3}\pi a^3\,\mathrm{d}n_{\rm pop}(a), \tag{2}$$

with a in cm and $\mathrm{d}n$ already carrying the bin width, so the opacity it produces refers to the same grains the emission does. Being a property of the model it is independent of wavelength and temperature: evaluate it once. A population whose builder states no density ($\rho_{\rm bulk} = 0$) is skipped rather than given a guessed one, and a model in which no population states a density returns 0.

The densities are part of each model's definition, not a free choice — the size distributions were normalized to element abundances with them:

model	population	$\rho_{\rm bulk}$ [g cm^{-3}]
astrodust	astrodust grains	2.74 (HD23 §2.3.2, already $\times(1-P)$)
	PAH	2.0 (HD23 $N_{\rm C} = 417\,(a/{\rm nm})^3$)
DL07	astrosilicate	3.5 (DL07 §2)
	carbonaceous	2.2 (graphitic carbon, DL07 §2)
Zubko, file-defined	each component	declared in its own optics table

Both DL07 values are the ones its own paper states, in the §2 paragraph that defines the effective radius $a = (3M/4\pi\rho)^{1/3}$: "amorphous silicate is assumed to have a mass density $\rho = 3.5\,{\rm g\,cm^{-3}}$, and carbonaceous grains are assumed to have a mass density due to graphitic carbon alone of $\rho = 2.2\,{\rm g\,cm^{-3}}$". The graphite value is deliberately *not* the 2.24 of WD01/LD01, which is the density implied by the carbon-atom count $N_{\rm C} = 470\,(a/{\rm nm})^3$ this builder selects; DL07 kept that count and quoted the rounder density, and the model built here is DL07's.

The DL07 carbonaceous grains are one continuous material sequence — PAH-like when small, graphite-like when large, joined by the DL07 $\xi(a)$ weight — with no size at which the solid changes, so the graphite density is used across the whole sequence rather than split at some radius. That the astrodust model's PAHs use 2.0 instead is HD23's convention for a different material in a different model, not a disagreement about graphite.

The same number normalizes the $K_{\rm abs}$ column of every `data/kext_*.dat` product (§8.), so a host that reads the tables and a host that links the library get the identical conversion. The values are 1.184949×10^{-26} g/H for astrodust, 1.750186×10^{-26} g/H for DL07 and 1.443932×10^{-26} g/H for Zubko BARE-GR-S.

Which DL07 normalization is followed, and why it matters. DL07's own Table 3 gives $M_{\rm dust}/M_{\rm H} = 0.0104$ for model $j_M = 7$ (MW3.1_60) — the very size distribution `build_dl07` selects — and the densities above reproduce it to about half a percent (1.750186×10^{-26} g/H, i.e. $M_{\rm dust}/M_{\rm H} = 0.01046$ with $m_{\rm H} = 1.6737 \times 10^{-24}$ g, +0.55%). Draine's tabulated optics for the same model, `kext_albedo_WD_MW_3.1_60_D03.all_2003`, state $M_{\rm dust}/N_{\rm H} = 1.870 \times 10^{-26}$ g/H in the file header, i.e. $M_{\rm dust}/M_{\rm H} = 0.0112$, which is 7% away from that author's own Table 3. The same file renormalizes the silicate distribution by 0.93 where the paper's Fig. 11 caption says 0.92.

SEDust takes the *optics* from the file — reproducing it is the point, and our $C_{\rm ext}$/H agrees with it to 0.1–0.3% over its whole range — and the *mass* from the paper. The two normalizations differ, so about 1% remains between our $K_{\rm abs}$ column and the file's; that residual is on Draine's side, not a free parameter here.

Choosing the table. Every builder takes an optional `kext_path` naming the file. Omitting it takes that model's default:

builder	default table
`build_astrodust`	`../data/kext_astrodust_MW_euv.dat`
`build_dl07`	`../data/kext_dl07_MW_euv.dat`
`build_zubko`	`../data/kext_zubko_BARE_GR_S.dat`
`build_from_files`	none

The astrodust and DL07 defaults are the EUV products because they are the *widest* grid each model has ($\sim 10^{-4}$–39810 μm): one file then serves a host that transports ionizing radiation and a host that does not, since the latter's grid is a subset of the former's, on the same nodes. Naming a `kext_path` is therefore how you *narrow* the table (to `kext_astrodust_MW.dat` or `kext_dl07_MW.dat`), or point at a product of your own. A file-defined model has no default at all: its product is named after the model, which only the descriptor knows, so a host that wants extinction out of such a model must name the file.

A `kext_path` that cannot be read *fails the build* (§4.5 for the code, which differs by builder) — a host naming a file that is not there is a configuration error. A *default* that cannot be read does not fail the build: the emission-only drivers must still run, and `calc_kext.x` builds a model precisely in order to write the table that is not there yet. In that case `m%kext_n` is 0 and `dust_extinction` reports status `2`.

The table is read at *build* time, not at query time, because these paths are relative to `sed/`. A host that changes directory around the `build_*` call — which is what MoCafe does — would otherwise find them broken by the time it asks for opacity.

Interpolation, and when it is exact. `Cext`, `Cabs` and `Csca` are interpolated linearly in $\log\lambda$–$\log C$; `gbar` linearly in $\log\lambda$, because it is signed and changes sign in the far infrared of some models. A wavelength of `m%lam` that coincides with a table node (within 10^{-6} relative, the tolerance that absorbs the round trip of λ through a decimal file) takes the tabulated value *unchanged*. Nothing is extrapolated: a grid running outside the table is refused with status `3`, not served a frozen boundary value.

An unextended astrodust or DL07 model therefore gets its numbers back bit-for-bit: its 1129 wavelengths are the T-matrix Q-table grid, which is exactly the long-wavelength part of the default EUV table (verified: all four scalar outputs bit-identical at all 1129 nodes; `albedo` differs by $\leq 10^{-12}$ relative because it is recomputed as $C_{\rm sca}/C_{\rm ext}$ rather than read). A model built with `lam_min` does *not* share the table's nodes below 0.0912 μm, since a different floor gives a different log spacing, and there the interpolation costs up to $\sim$1% in $C_{\rm ext}$ (measured 1.2×10^{-2} at worst, near 0.023 μm, for `lam_min` = 0.0124 μm). A host that needs the ionizing band at full fidelity should generate a table at its own `lam_min` (`./calc_kext.x astrodust <lam_min>`) and pass it as `kext_path`.

Every model scatters. All four builders populate the extinction-side scattering optics, so `albedo` and `gbar` are physical for every model:

- **astrodust**: $C_{\rm sca}$ and $\langle\cos\theta\rangle$ from the random-orientation T-matrix Q table (and, below 0.0912 μm, from Mie on the same dielectric function).

- **DL07**: Mie on the D03 astrosilicate and graphite dielectric functions (`q_silicate_full` / `q_graphite_full`), on the same random-orientation average ($\frac{1}{3}E \parallel c + \frac{2}{3}E \perp c$) as the absorption. The two carbonaceous charge states share one scattering description: charge shifts PAH absorption features, not the graphite sphere's scattering.
- **Zubko / file-defined**: the $Q_{\rm sca}$ and g columns of the model's own DustEM tables, i.e. the authors' own multilayer-sphere solution, read from the same file as $Q_{\rm abs}$.

A population that genuinely does not scatter leaves its scattering optics unallocated and contributes zero. In the astrodust model the PAHs are exactly that case — PAH scattering is Rayleigh-negligible — so they enter extinction through absorption only, and the astrodust grains carry all the scattering and the asymmetry.

Polarized optics are astrodust only. `Cpol_ext` and `Cbir_ext` come back as zeros from `build_dl07`, `build_zubko` and `build_from_files`, which carry no polarized optics at all, and from an astrodust model built with `load_polarized_optics = .false.` `dust_has_polarized_optics(m)` distinguishes the two cases.

6. Polarized dust optics

The library exports two polarized quantities for the HD23 astrodust model: a polarized *emission* spectrum through `dust_emission`, and a polarized *extinction* cross section through `dust_extinction` or, equivalently, through the pre-computed extinction table. Both are *intrinsic*: the size integral and the alignment weighting are done, but the geometry of the local magnetic field is not applied. What remains for the calling code is set out in §6.6.
This section documents the interface only. For the alignment physics, the polarized transfer equation, and the sign and index conventions behind these quantities, see the companion report `aligned_grain_polarization.pdf` (§3 for the optical properties, §4 for alignment, §5 for transfer).

Wavelength coverage. `Cpol`, `Cpol_ext` and `Cbir_ext` come from an orientation-resolved table that covers all 1129 wavelengths of the Q-table grid, 0.0912–39810 μm, so nothing has to be regenerated to use them anywhere in that range. The *default* table is the HD23 release file `data/dielectric/q_DH21Ad_P0.20_Fe0.00_1.400.dat.gz`; SEDust's own regenerated table, `tmatrix/output/q_astrodust_jori_P0.20_Fe0.00_1.400.dat.gz`, is opt-in through `qpol_path` and is the one that carries the 4th (birefringence) block, so `Cbir_ext` is zero with the default and nonzero with it (§6.4). The angle-resolved *scattering* matrix is a separate, coarser product — five optical bands (§6.5) — and shortward of 0.0912 μm the polarized extinction has an announced hole (§7.3).

6.1 Polarized emission

```
call dust_emission(m, J_lam, lamI_total [, lamI_chan] [, status] [, lamI_pol])
```

- `lamI_pol(m%NLAM)` (optional): the intrinsic polarized emission, with the same shape and the same units as `lamI_total`. It is the emission a population of perfectly aligned grains would radiate with their symmetry axes in the plane of the sky.

- Only populations that carry both polarized optics and an alignment efficiency contribute. PAHs are taken to be unaligned by HD23 and add zero, as does any model built without polarized optics.
- The argument is optional and comes last, so every existing call site remains valid.

The polarized channel is accumulated at every point where the total emission is accumulated, using the same grain temperature weights, so a stochastically heated grain contributes to both consistently under all four values of `m%stoch_method`.

6.2 Polarized extinction

Two routes give the same quantity. The direct one is the optional `Cpol_ext` argument of `dust_extinction` (§5.),

```
call dust_extinction(m, Cext, Cabs, Csca, gbar, Cpol_ext)
```

which returns $\sum_a \mathrm{d}n_{\mathrm{Ad}}(a) C_{\mathrm{pol,ext}}(\lambda, a)\, f_{\mathrm{align}}(a)$ [cm^2/H] on `m%lam`. Prefer it in a code that already holds a model object. Unlike the four scalar outputs of that call, this one is computed from the model's optics rather than read from a table, so it follows the alignment efficiency the host has set (§6.3) — a tabulated column could not.

The second route is the file `data/kext_astrodust_MW.dat`, written by `calc_kext.x astrodust`, see §8., which carries the dichroic extinction as an eighth column (Table 3). It is the right choice for a tool that wants the tabulated optics without linking the library, and it is valid only for the HD23 alignment fit it was written under. Codes that read only the first seven columns are unaffected by its presence, and `dust_extinction` is one of them: it reads no polarized column from any table.

col.	quantity	unit	note
1	`lambda`	μm	wavelength grid
2	`albedo`		$C_{\mathrm{sca}}/C_{\mathrm{ext}}$
3	`<cos>`		scattering asymmetry g
4	`C_ext/H`	cm^2/H	total extinction
5	`C_abs/H`	cm^2/H	total absorption
6	`C_sca/H`	cm^2/H	total scattering
7	`K_abs`	cm^2/g	mass absorption coefficient
8	`C_polext/H`	cm^2/H	dichroic extinction (new)

Table 3: Columns of `kext_astrodust_MW.dat`. Columns 1–7 are written for every model; column 7 is $C_{\mathrm{abs}}/\mathrm{H}$ divided by that model's dust mass per H, Eq. (2). Column 8 is $\sum_a \mathrm{d}n_{\mathrm{Ad}}(a) C_{\mathrm{pol,ext}}(\lambda, a)\, f_{\mathrm{align}}(a)$, the maximum polarized extinction, quoted before any geometric factor, and this file is the only product that carries it.

6.3 Changing the alignment efficiency

By default the alignment efficiency is the HD23 fit. Two setters, both re-exported by `dust_lib`, replace it on a model that already exists.

```
call dust_set_alignment(m, f_max, a_align, alpha_align [, status])
call dust_set_alignment_profile(m, aeff_in, falign_in [, status])
```

The first installs the HD23 power-law rolloff

$$f_{\rm align}(a) \;=\; \frac{f_{\rm max}}{1+(a_{\rm align}/a)^{\alpha_{\rm align}}}, \tag{3}$$

with $a_{\rm align}$ in μm. The second installs an arbitrary tabulated profile: the caller supplies (`aeff_in`[μm], `falign_in`) pairs and each population's efficiency is interpolated from them onto its own radius grid, linearly in $\log a$ and clamped at the ends of the table. This is the route for prescriptions Eq. (3) cannot express: a RAT-derived Rayleigh reduction factor $R(a)$, or the GRADE-POL exponential $f_{\rm max}[1-\exp(-(0.5a/a_{\rm align})^3)]$.
Both refill `falign` only for populations that carry polarized optics. PAHs, which HD23 take to be unaligned, are left untouched and keep contributing zero.

No temperature solve is repeated. This is the property worth knowing before designing a parameter study. The alignment efficiency enters as a weight in the size integral, *outside* the temperature solution: it multiplies the polarized accumulator and never enters $P(T)$. Changing the alignment therefore does not invalidate any temperature distribution, and `lamI_total` is bit-for-bit unchanged across a setter call. A scan over alignment parameters costs one `dust_emission` call per point, not one model build.

Negative efficiencies are allowed on purpose. `dust_set_alignment_profile` requires `falign_in` to lie in $[-1,1]$ and *rejects* rather than clamps a value outside it, since $|f|\le 1$ is the physical bound on a Rayleigh reduction factor and a value beyond it is a caller error that silent clamping would hide. Negative values inside the range are accepted: a grain with slow internal relaxation has $Q_X=-0.1$, a negative reduction factor that flips the polarization direction by 90°. That is the accepted origin of polarization parallel to **B** at millimeter wavelengths and perpendicular to it in the submillimeter, and clamping at zero would silently delete it. `dust_set_alignment` keeps $f_{\rm max}$ in $[0,1]$, the power law having no such sign freedom.
With `status` present a bad argument sets it nonzero and returns; without it, the run stops. The codes are 1, 2, 3 in the order the arguments are listed above for `dust_set_alignment` ($a_{\rm align}\le 0$, $\alpha_{\rm align}\le 0$, $f_{\rm max}$ outside $[0,1]$), and for `dust_set_alignment_profile` 1 for mismatched or too-short arrays, 2 for an `aeff_in` that is not positive and strictly increasing, and 3 for a `falign_in` value outside $[-1,1]$.

Status code 4: a loaded aligned scattering table is now stale. If an aligned scattering table has been loaded (§6.5) and the requested profile differs from the one that table was integrated under, either setter returns the new code 4. This is *non-fatal*: the $f_{\rm align}$ update completes and the emission and `Cpol_ext` channels re-integrate live under the new profile, so those outputs always honor the latest alignment. The code only signals that the loaded scattering matrices no longer correspond to it, because they were size-integrated once under the recorded profile. The sanctioned runtime variation of the aligned scattering optics is the scalar η (§6.5); a genuinely different profile requires regenerating the table with `run_scatmat_aligned.x profile=FILE`. A tabulated profile never matches the recorded analytic one and so always raises the flag.

6.4 Where the polarized optics come from

The orientation-resolved spheroid cross sections behind `lamI_pol` and `Cpol_ext` default to the Draine & Hensley (2021) release table. They can instead be taken from a table SEDust regenerates from the astrodust dielectric function with its own T-matrix engine: `build_astrodust` and `sed_init` accept an optional `qpol_path` (with `qpol_wave_path` and `qpol_aeff_path` for its grid axes) that overrides the default file. The regenerated table matches the release to a few parts in 10^4 where grains carry weight and is written in the release table's own format, so nothing else in the call changes. Its one deliberate benefit is that it supplies the polarized cross sections from the same exact optics as the total intensity, removing the mixed averaging convention the release forces; see `sedust_polarization_implementation.pdf` §7 and `tmatrix/run_q_jori.x`. Leaving `qpol_path` unset keeps the release table and the existing results, though an explicit `qpol_path` that cannot be read now fails the build with `status = 4` (and an unconditional stderr message), whereas the implicit default still degrades gracefully to an unpolarized model.

Scalar-only builds. `build_astrodust` and `sed_init` take an optional `load_polarized_optics`. Absent or `.true.` is the present behavior. `.false.` never opens the orientation-resolved polarized Q table: `Cpol`, `Cpol_ext`, `Cbir_ext` and `falign` stay zero, and the scalar `Cext`/`Cabs`/`Csca`/`gbar` and the total SED come back bit-identical to a polarized build at zero alignment. Combining it with an explicit `qpol_*` or `scatmat_path` is a contradiction and returns `status = 5`. The query `dust_has_polarized_optics(m)` reports whether the built model carries polarized optics at all; the aligned-scattering table's load state is separately visible as `scm_loaded`.

6.5 Aligned-grain scattering optics

For the astrodust spheroid SEDust also supplies the angle-resolved scattering optics of the aligned population, for a polarized RT code that transports a Stokes vector: the fixed-orientation Mueller matrix, the 4×4 extinction matrix, and the random-orientation remainder. These come from a table written by `tmatrix/run_scatmat_aligned.x` (five optical bands ship with SEDust); how it is built is documented in `sedust_polarization_implementation.pdf` §8. The API follows the library's own lifecycle — initialize once, then query along the photon path — split into two strictly separated layers.

Loading (once per run, serial). Pass the table path as an optional `scatmat_path` to `sed_init` or `build_astrodust`, exactly like `qpol_path`:

```
call build_astrodust(m, qtab, sizedist, NT, T_lo, T_hi, status=st, &
                  qpol_path=QPOL, scatmat_path=SCATMAT)
```

The production table holds five optical bands — 0.36, 0.44, 0.55, 0.64 and 0.79 μm, approximately $UBVRI$ — on a grid of 19 incidence angles $\theta_i \times$ 181 scattering angles $\theta_s \times$ 37 azimuths ϕ. Five bands is a deliberate choice, not a limitation of the format: polarized transfer is almost never wanted at every wavelength, and what matters is that any other wavelength *can* be produced (§8.7 — note the caveats there, since the scattering table is the less flexible of the

two products). At run time the queries are a trilinear interpolation on that grid, a few hundred floating-point operations per scattering event, and `scatmat_band` maps a requested wavelength onto a stored band once.

The table is parsed and its size integrals kept in memory (about 81.5 MB for the production grid, growing linearly with the number of bands). A `scatmat_path` that fails to load is an error (`status = 3` from the builder), because a host that asked for the table depends on it. A gzipped table — both this and the polarized Q table ship `.gz` — is decompressed to a scratch copy under `TMPDIR` (else `/tmp`) named with the process id (`scatmat_aligned_<pid>.dat`, `q_jori_<pid>.dat`) and deleted after the read, so concurrent MPI ranks never collide and a read-only launch directory is never written. This layer is not thread-safe; call it from serial code before the photon loop.

Path queries (per event, thread-safe). Six routines, all re-exported by `dust_lib`, are pure reads of the loaded arrays: they write only their own output arguments, do no I/O and no allocation, and are safe to call concurrently from OpenMP photon threads. The whole cell dependence enters through two scalars the host already holds — the alignment scale η, and the incidence angle $\theta_i = \arccos(\hat{k}\cdot\hat{B})$ between the photon direction and the local field, from one dot product.

```
call scatmat_band(lambda_um, iband, exact) ! pick the band once
call extinction_matrix_aligned(iband, theta_i, eta, kmat) ! 4x4 [um^2/H]
call mueller_matrix_total(iband, theta_i, theta_s, phi, eta, z) ! 4x4 [um^2 sr^-1/H], absolute
call mueller_matrix_aligned(iband, theta_i, theta_s, phi, z) ! 4x4 [um^2 sr^-1/H], aligned only, eta=1
call mueller_matrix_random(iband, big_theta, f_tot, f_ref) ! 6-elt random matrices
call scattering_cross_sections(iband, theta_i, eta, csca_aligned, csca_unaligned)
```

- `scatmat_band`: `lambda_um` [μm] in; `iband` the nearest stored band; `exact .true.` if it matched to 10^{-3} fractionally. The hot path then takes `iband` so no wavelength search runs each event.
- `extinction_matrix_aligned`: `kmat(4,4)` [μm^2/H], the η-scaled aligned extinction matrix at incidence `theta_i` [deg]. Diagonal $\eta C_{\rm ext,al}$; $K(1,2) = K(2,1) = \eta C_{\rm pol,al}$ (dichroism); $K(3,4) = \eta C_{\rm bir,al}$, $K(4,3) = -\eta C_{\rm bir,al}$ (birefringence). `theta_i` in $[0,180^\circ]$ is folded to $[0,90^\circ]$, K being even under the equatorial reflection.
- `mueller_matrix_total` (**recommended**): the *absolute* combined phase matrix `z(4,4)` [μm^2 sr^{-1}/H] at grain-frame geometry and alignment η, with both populations already assembled,

$$z = \eta Z_{\rm al} + L(\pi-\sigma_2)\,F_{\rm unal}(\Theta)\,L(-\sigma_1)\,/\,(4\pi), \quad F_{\rm unal} = C_{\rm sca,tot}F_{\rm tot} - \eta C_{\rm sca,ref}F_{\rm ref},$$

  with $\cos\Theta = \cos\theta_i\cos\theta_s + \sin\theta_i\sin\theta_s\cos\phi$ and the meridional rotation angles (closed forms, poles included)

$$\sigma_1 = \mathrm{atan2}(\sin\theta_s\sin\phi,\ \cos\theta_i\sin\theta_s\cos\phi - \sin\theta_i\cos\theta_s),$$
$$\sigma_2 = \mathrm{atan2}(-\sin\theta_i\sin\phi,\ \cos\theta_i\sin\theta_s - \sin\theta_i\cos\theta_s\cos\phi).$$

  L is the Stokes rotation. Prefer this call to reconstructing the mixture from the public arrays: it is the one place the normalization and the rotation signs are certified.
- `mueller_matrix_aligned`: the aligned matrix alone, `z(4,4)` [μm^2 sr^{-1}/H] at the reference alignment ($\eta = 1$; scale it yourself). `theta_i`, `theta_s` in $[0,180^\circ]$, `phi` in $[0,360^\circ]$; the stored octants ($\theta_i \le 90^\circ$, $\phi \le 180^\circ$) and the rest are handled by interpolation plus the header's symmetry relations.

- `mueller_matrix_random`: `f_tot(6)`, `f_ref(6)`, the two six-element $(F_{11}, F_{22}, F_{33}, F_{44}, F_{12}, F_{34})$ random-orientation matrices at `big_theta` [deg], α_1-normalized as stored ($\frac{1}{2}\int F_{11}\,\mathrm{d}\cos\Theta = 1$, i.e. $\int F_{11}\,\mathrm{d}\Omega = 4\pi$). The absolute differential matrix is $C_{\rm sca}F/(4\pi)$: restore $\mu\mathrm{m}^2\ \mathrm{sr}^{-1}$/H with `f_tot*scm_csca_tot(iband)/(4*PI)` and `f_ref*scm_csca_ref(iband)/(4*PI)`, the $1/(4\pi)$ turning the 4π-normalized F into the matrix whose element 11 integrates to $C_{\rm sca}$ over the sphere. (Omitting it overweights the remainder by 4π relative to $Z_{\rm al}$, whose stored matrix already integrates to $C_{\rm sca,al}$.)
- `scattering_cross_sections`: `csca_aligned` $= \eta C_{\rm sca,al}(\theta_i)$ and `csca_unaligned` $= C_{\rm sca,tot} - \eta C_{\rm sca,ref}$ [$\mu\mathrm{m}^2$/H], for the albedo and the population split. An optional fourth output `csca_pol_aligned` $= \eta C_{\rm sca,pol,al}(\theta_i)$ is the polarized scattering cross section $\int Z_{12}\,\mathrm{d}\Omega$ of the aligned matrix (Peest et al. 2023), which the polarization-dependent albedo $\Lambda = [C_{\rm sca} + C_{\rm sca,pol}\,Q/I]/[C_{\rm ext} + C_{\rm pol}\,Q/I]$ needs; the random remainder contributes zero to it by azimuthal symmetry.

The η contract, and unit note. A cell's alignment enters only as η: the aligned matrix and the aligned K scale as η, and the unaligned remainder scattering matrix is $Z_{\rm unal} = [C_{\rm sca,tot}F_{\rm tot} - \eta C_{\rm sca,ref}F_{\rm ref}]/(4\pi)$ in absolute $\mu\mathrm{m}^2\ \mathrm{sr}^{-1}$/H, whose element-11 solid-angle integral is $C_{\rm sca,tot} - \eta C_{\rm sca,ref}$. For the total extinction matrix the aligned population must not be counted twice: the isotropic `Cext` from `dust_extinction` already contains its reference-orientation average $C_{\rm ext,ref}$, so assemble

$$K_{\rm total} = (C_{\rm ext}^{\rm iso} - \eta C_{\rm ext,ref})\,I + \eta K_{\rm al}(\theta_i), \tag{4}$$

which returns $C_{\rm ext}^{\rm iso}$ on the diagonal at the incidence average and adds the direction-resolved dichroism and birefringence. These aligned-scattering quantities are in $\mu\mathrm{m}^2$ (per H), whereas `dust_extinction` returns cm^2/H; convert with a factor 10^8 before combining them, as Eq. (4) requires.

Inlining. The loaded grids and integrals are also exposed `public, protected` (`scm_lambda`, `scm_theta_i`, `scm_theta_s`, `scm_phi`, `scm_theta_ran`, `scm_cext_tot`, `scm_csca_tot`, `scm_cext_ref`, `scm_csca_ref`, `scm_F_tot`, `scm_F_ref`, `scm_Z`, and the `scm_n*` sizes), so a host that prefers to inline its own lookups can take them once at startup and never call back. A complete two-cell reference consumer of every call is `sed/rt_example/use_dustlib_scatmat.f90` (`make use_dustlib_scatmat.x`); it demonstrates the band selection, Eq. (4), the aligned and random queries, and the closure checks.

6.6 Division of labor

Table 4 is the part to get right. SEDust stops at the intrinsic quantity; everything that depends on where the field points is left to the transfer code, because SEDust knows nothing about the cell geometry.

The observed Stokes emissivities follow from the exported quantity as

$$(j_Q,\, j_U) \;=\; \texttt{lamI\_pol}\,\sin^2\gamma\, F_{\rm turb}\,(\cos 2\phi,\, \sin 2\phi), \tag{5}$$

with j_I = `lamI_total` unchanged. The same $\sin^2\gamma$ and $F_{\rm turb}$ multiply `Cpol_ext` (column 8) when the extinction matrix is assembled. Equation (5) is stated here as a usage rule; its derivation is in §5.1 of `aligned_grain_polarization.pdf`.

Done by SEDust	Left to the calling code
Integral over the grain size distribution	$\sin^2\gamma$, with γ the angle between the *local magnetic field* and the *line of sight* (not a sky-plane angle)
Weighting by the alignment efficiency $f_{\rm align}(a)$	Turbulent depolarization $F_{\rm turb}$
Summation over components (only astrodust is aligned; PAHs contribute zero)	Splitting into Q and U using the position angle ϕ of the projected field

Table 4: What is already integrated into `lamI_pol` and into `Cpol_ext` (equivalently, column 8), and what a radiative transfer host must still apply.

6.7 Minimal example

```
use dust_lib
use radfield, only: J_Mathis
type(dust_model_t) :: m
real(wp), allocatable :: J_lam(:), lamI_total(:), lamI_pol(:), p(:)
integer :: i

call build_astrodust(m, qtab, sizedist, 200, 2.7_wp, 5.0e3_wp)
allocate(J_lam(m%NLAM), lamI_total(m%NLAM), lamI_pol(m%NLAM), p(m%NLAM))

call J_Mathis(1.0_wp, m%lam, J_lam)
call dust_emission(m, J_lam, lamI_total, lamI_pol=lamI_pol)

! intrinsic polarization fraction (before any geometric factor)
do i = 1, m%NLAM
   if (lamI_total(i) > 0.0_wp) then
      p(i) = lamI_pol(i) / lamI_total(i)
   else
      p(i) = 0.0_wp
   end if
end do
```

Multiply p by $\sin^2\gamma\, F_{\rm turb}$ to obtain the polarization fraction of a real line of sight.

6.8 Limits

- **Astrodust only.** `build_dl07` and `build_zubko` have no polarized optics. `build_dl07` releases any polarized arrays left by a previous astrodust build, so `lamI_pol` returns zeros rather than stale values.
- **PAHs contribute zero** ($f_{\rm align} = 0$, an HD23 assumption). The polarized emission therefore vanishes in the mid infrared, where stochastically heated PAHs dominate.

- **Circular polarization: available only from the regenerated table.** The released optics table carries $Q_{\rm ext}$, $Q_{\rm abs}$ and $Q_{\rm sca}$ but no phase retardation, so with it $C_{\rm circ} = 0$. The regenerated orientation-resolved table (§6.4) adds a birefringence block, and `dust_extinction` returns it as the optional `Cbir_ext` (§5.); the aligned scattering table carries the same birefringence in its $K(\theta_i)$ (§6.5). Consuming it in the $U \leftrightarrow V$ transfer term is the host's task.
- **Scattering by aligned grains: now available.** The angle-resolved Mueller matrix of the aligned astrodust spheroid, which no released file contains, is computed by `run_scatmat_aligned.x` and served through the API of §6.5. The release-derived cross sections still supply only the integrated $Q_{\rm sca}$; the angular optics come from the T-matrix engine.
- **Far-infrared and submillimeter polarized emission is complete without a scattering matrix.** The albedo there is essentially zero, so the transfer needs only the extinction matrix and the emission vector, both fully determined by what is exported here.

Reference numbers. For a Mathis ISRF the intrinsic polarization fraction `lamI_pol/lamI_total` is 0.00% at 12 μm, 13.67% at 100 μm, 17.15% at 154 μm, 18.79% at 350 μm and 19.15% at 850 μm. Column 8 reproduces the released `polarized_extinction.dat` to a median of 0.0296%.

7. The extreme-ultraviolet extension

The astrodust model's wavelength grid is the T-matrix Q table's, which stops at 0.0912 μm — 13.6 eV, the Lyman limit. A host that transports ionizing radiation needs shorter wavelengths, so both `build_astrodust` and `build_dl07` take an optional `lam_min` [μm]:

```
call build_astrodust(m, qtab, sizedist, NT, T_lo, T_hi, lam_min=1.001e-4_wp)
```

Log-spaced points are then prepended from `lam_min` up to the table, at a step no coarser than the table's own $\Delta \ln \lambda = 0.01156$ at its short-wavelength end, with `m%lam(1)` set to `lam_min` exactly. **Omitting `lam_min` prepends nothing and leaves the model bit-identical to before.**

7.1 Where the optics come from in the extended band

- **Astrodust grains.** Bohren–Huffman Mie for the *volume-equivalent sphere* on the Draine & Hensley (2021) dielectric function — the same material as the Q table, but not the same shape. See §7.2.
- **PAH / carbonaceous.** Above 21.4 eV ($1/\lambda = 17.25$ μm^{-1}) the DL07 PAH cross section is zero by construction, so graphite is the *whole* carbonaceous absorption there. Below that energy the D16 turbostratic-graphite sphere table is used as before; above it the code switches to Mie on the D03 graphite dielectric functions (random orientation, $\frac{1}{3}$ parallel $+\frac{2}{3}$ perpendicular), because the D16 table's radius axis bottoms out at 10^{-3} μm while the astrodust size grid starts at 3.55×10^{-4} μm. In the Rayleigh limit $Q_{\rm abs} \propto a$, so clamping a smaller grain to the table's first radius overestimates its absorption by 10^{-3} μm$/a$ — up to 2.4× for the smallest PAH the size distribution

populates ($4.22 \times 10^{-4}\,\mu$m). *This branch is unreachable on the unextended grid*: its shortest wavelength gives $1/\lambda = 10.96 < 17.25$, so every previous result is unaffected.

- **DL07 model.** Its optics are Mie on the D03 dielectric functions at *every* wavelength, so nothing changes character across the band and the extension is a grid extension only.
- **Zubko model.** No extension, and none possible: its tabulated optics already span 10^{-3} to $10^{4}\,\mu$m, and those tables *are* the model definition, so there is no dielectric function to extend them with.

7.2 The shape approximation, and why it is bounded

The Q table is a random-orientation T-matrix solution for an oblate spheroid of axial ratio $b/a = 1.4$; the extended band uses the volume-equivalent *sphere*. The justification is **not** the anomalous-diffraction argument $|m-1| \ll 1$: for this material $|m-1| = 0.765$ at the seam and still 0.564 at 20 eV. It rests instead on the two limits that actually apply:

- **Small grains are in the Rayleigh limit** ($x = 2\pi a/\lambda = 0.033$ at the seam for the smallest populated radius, and only 0.24 at 100 eV). There the ellipsoid polarizability is $\alpha_j = V(\epsilon-1)/\{4\pi[1+L_j(\epsilon-1)]\}$, so shape enters *only* through $L_j(\epsilon-1)$. And $|\epsilon-1|$ falls from 1.86 at 13.5 eV to 1.10 at 20 eV, 0.28 at 40 eV and 0.061 at 100 eV: the depolarization factors drop out and sphere and spheroid converge on the same $C_{\rm abs}$.
- **Large grains are in the geometric-optics limit**, where $C_{\rm ext} \to 2\times$(orientation-averaged projected area). By Cauchy's theorem that average is $S/4$, and for $b/a = 1.4$ the spheroid's $S/4$ exceeds $\pi a_{\rm eff}^2$ of its volume-equivalent sphere by 2.12%. Mie is therefore low by $1-1/1.0212 = 2.08\%$ in that limit. **The error converges to a constant, not to zero**, however far into the EUV one goes.

Consequently the error does *not* fall monotonically with photon energy. Only $a \lesssim 2 \times 10^{-3}\,\mu$m improves monotonically; $a \gtrsim 0.01\,\mu$m gets *worse* between 13.6 and 30 eV before recovering, and the largest sizes settle at a nearly energy-independent −1.9 to −2.0%. Over the whole band and the whole size range the error stays within about 2%; `sed/src/q_astrodust.f90` tabulates it size by size.

Seam continuity. In the size-integrated curve the step across $0.0912\,\mu$m is −1.54% in $C_{\rm ext}$ and −1.90% in $C_{\rm abs}$ — *smaller* than the neighboring steps of the table's own grid (−1.95% and −1.72% for $C_{\rm ext}$; −2.20% and −1.94% for $C_{\rm abs}$), so absorption and extinction join without a visible discontinuity. Scattering does show one: $C_{\rm sca}$ steps by +0.74% where its neighbors step by −0.39% and −0.36%, and the albedo by +2.31% where its neighbors give +1.59% and +1.39% — a seam of roughly one percentage point, consistent with the 2% shape error being carried mostly by $Q_{\rm sca}$.

7.3 The polarized optics do not extend with the grid

This is the one place where the polarized branch has a hole, and it is announced rather than hidden. The dichroic and birefringent cross sections come from the orientation-resolved table, which covers the Q table's own 0.0912–$39810\,\mu$m and no further. When `lam_min` carries the grid below that, `build_Cpol` looks for the EUV companion table `q_astrodust_jori_euv_P0.20_Fe0.00_1.400.dat.gz`. **That table does not ship**, so what normally happens is:

```
build_Cpol: no EUV polarized table (...)
          dichroic extinction and birefringence are zero below 9.120E-02 um
          (zero by omission, not by physics).
```

The message goes to `error_unit` unconditionally, and the EUV block of `Cpol_ext` and `Cbir_ext` is left at zero. **A host must not read that zero as a physical result.** Measured from first principles (`tmatrix/driver/euv_polarized_optics.f90`), the alignment-weighted, size-integrated $|C_{\rm pol,ext}|/C_{\rm ext}$ is 1.26×10^{-3} at the $0.0912\,\mu$m seam, *rises* to 3.64×10^{-3} near 20.6 eV — a factor 2.9 — and decays to 1.6×10^{-4} at 100 eV, **with the sign opposite to the optical band**; the reversal falls near $0.106\,\mu$m. A sphere has exactly zero dichroism and exactly zero birefringence, so the scalar EUV optics could not have supplied these entries even in principle.

To fill the band, generate the companion table with `tmatrix/run_q_jori.x` (§8.) and pass the resulting pair as `qpol_euv_path` and `qpol_euv_wave_path`. With a table present, `build_Cpol` interpolates it in $\log\lambda$ and $\log a$, and a grid running outside its 0.0124–0.0912 μm coverage is refused (`status = 9`) rather than answered with an invalid limit. That table stops at $0.0124\,\mu$m (100 eV) because the opaque geometric-optics limit it leans on needs the chord optical depth $4\,\mathrm{Im}(m)\,x$ to be large, and for astrodust that is 3.7 at $x = 50$ there and falls below 1 by 200 eV.

The *scalar* optics are unaffected by any of this: $C_{\rm ext}$, $C_{\rm abs}$, $C_{\rm sca}$, `gbar` and `albedo` are complete over the whole extended grid.

7.4 The single-photon ceiling comes from the field

The stochastic solvers set the top of a grain's enthalpy window from the energy of the hardest single photon it can absorb. That ceiling used to be the hard-coded Lyman limit, 13.6 eV, which an ionizing band makes wrong: a photon harder than the ceiling drives an excursion the window cannot hold. It is now

$$\max\bigl(13.6\,\mathrm{eV},\ hc/\lambda_{\rm hard}\bigr), \qquad \lambda_{\rm hard} = \min\bigl\{\lambda_i\ :\ J_\lambda(\lambda_i) > 10^{-12}\,\max_j J_\lambda(\lambda_j)\bigr\},$$

computed by `hardest_photon_energy` in `sed/src/radfield.f90` and called from all three places that need it — `sed_grain_loop` and `narrow_iterative` in `sed_astrodust.f90`, and `qm_solve_grain` in `stoch_qm.f90`. Any units may be passed for J_λ, since only ratios within the array are used.

The ceiling is a property of the field, not of the grid. That distinction is the whole point, and a coincidence hides it. A model's wavelength grid is the grid of its optics tables, which can reach far past the band a transported field occupies:

model	grid floor	implied photon energy
astrodust	$0.0912\,\mu$m (T-matrix Q table)	13.595 eV
DL07	$0.0912\,\mu$m (the same table)	13.595 eV
Zubko	$1.000\times10^{-3}\,\mu$m (DustEM tables)	1239.8 eV

For astrodust and DL07 the grid floor *is* the Lyman limit, so 13.595 eV $<$ 13.6 eV: the maximum returns the constant, and grid and field give the same answer whichever one is measured. For Zubko they differ by a factor of 91, and the field is the one that is right — a Mathis field carries no photon at all below $0.0912\,\mu$m, however far the optics table extends.

Taking it from the grid costs resolution, not physics. `umax` sets the upper edge of a *fixed* number of enthalpy bins, so a ceiling 91 times too high starts the solve with bins 91 times too coarse. The refinement loops do contract the window once the top bin is unpopulated, but the contraction is guarded (`umax > 1.02*umaxlo` and `umax > 1.01*ub(jcut)`) and capped at `MAX_ITER = 10`, so it does not return to where it began; expansion carries no such guard. Measured in a host code on the Zubko model, with no `lam_min` requested at all: dust temperature +0.0705 K median and positive in every cell, the ten brightest SED bins −0.9 to −1.9%, the emission image 1.9% median and 5.4% maximum, total emitted energy −0.08%. A systematic redistribution at nearly conserved energy is what a coarser bin grid produces, and `docs/EUV_EXTENSION_HOST_REGRESSION.md` records the measurement.

Why 10^{-12} of the peak, and not simply $J_\lambda > 0$. Exact zeros must be excluded — `J_Mathis` returns exactly zero below $0.0912\,\mu$m — but an exact positivity test is too fragile: a host that hands over denormal or roundoff-level residue in its unilluminated bins would push the ceiling back up by orders of magnitude. A component a fraction 10^{-12} below the peak of J_λ contributes at most that fraction of the photon absorption rate; allowing two decades for the run of $C_{\rm abs}$ across the illuminated band, the enthalpy states it could populate carry probability $\lesssim 10^{-10}$ of the peak, at or below the 10^{-13} tail level (`PMIN_UP`, `PMIN_UP_QM`) at which both solvers already truncate the window. Such a component cannot change a resolved excursion, and 10^{-12} still sits eighteen decades above the residue it is meant to reject.

The error is one-sided on purpose. Both refinement loops *expand* the window while the top bin is still populated, so an underestimated ceiling is corrected as the loop runs; the guarded contraction can stop short of where it began, so an overestimate is not corrected at all. `MAX_ITER = 10` bounds the correction in either direction. Overestimating is therefore the dangerous direction, which is why the threshold is set high enough to reject junk rather than as low as floating point allows.

`main_zubko.x` exists to keep this path covered. The defect escaped `main_astrodust.x` and `main_dl07.x` because for those two models the grid floor and the field's short end are the same number: no driver in the tree ran the emission solvers on a model whose optics grid reaches past the illuminated band. `main_zubko.x` (§8.4) closes that gap and is built by the default `make`. It prints the two candidates side by side, so the difference shows in the first lines of output:

```
$ ./main_zubko.x # Mathis field, stops at the Lyman limit
 optics grid short end : 1.0000E-03 um -> 1.23984E+03 eV
 hardest photon in the field: 1.35946E+01 eV
 single-photon bound in use : 1.36000E+01 eV
$ ./main_zubko.x euv # a hard component added below the Lyman limit
 optics grid short end : 1.0000E-03 um -> 1.23984E+03 eV
 hardest photon in the field: 3.70141E+02 eV
 single-photon bound in use : 3.70141E+02 eV
```

Without `euv` the ceiling must stay at 13.6 eV however far the optics grid extends, and the Zubko SED is then identical to what the hard-coded constant produced; with `euv` it must follow the field up, and it does.

A model built without `lam_min` is unaffected. For astrodust and DL07 the maximum selects the 13.6 eV constant on the unextended grid either way, so those results are unchanged to the last bit.

7.5 What else the extension does, and does not, change

- **The Planck integrals are anchored to the table range.** The internal $\log\lambda$ integration grid for $\kappa_B(T,a)$ keeps its step and its sample points over the optics-table range and merely *prepends* points below it, so widening the model grid cannot coarsen the sampling of the range that carries the integrand. Measured: with a $1.001\times10^{-4}\,\mu$m floor, κ_B is unchanged to round-off over the thermal range ($\max|\Delta\kappa_B|/\kappa_B = 3\times10^{-16}$ for $T\le3000$ K, and exactly zero below ~2400 K); $\kappa_{\rm CMB}$ is bit-identical. The only nonzero difference is 1.1×10^{-8} at the very top of a 5000 K grid, where the Wien tail genuinely reaches into the extended band — that is real extra emission, not a sampling artifact. Without the anchoring, spending a fixed budget of points on the widened interval would have moved κ_B by $\sim10^{-4}$ for a purely numerical reason.
- **Stochastic heating by hard photons is not solved by this change.** The extension fixes the photon-energy mapping and the absorbed-energy accounting. It does not address the finite enthalpy grid: upward transitions above the top of that grid are accumulated into the highest temperature bin, and for the smallest PAHs a 54 eV photon already exceeds $H(5000\ \mathrm{K})$. Nor does it add photoionization, electron energy loss, fragmentation or PAH destruction. Treat the extended band as extinction and absorbed-energy physics, not as a validated hard-EUV PAH *emission* model.

7.6 How far each model can be checked

author-provided data	coverage	what it contains
Draine, `kext_albedo_WD_MW_3.1_60_D03` (DL07 / WD01)	10^{-4}–$10^{4}\,\mu$m = 1.24×10^{-4} eV–12.4 keV	λ, albedo, $\langle\cos\theta\rangle$, $C_{\rm ext}$/H, $K_{\rm abs}$, $\langle\cos^2\theta\rangle$
HD23 astrodust release (`extinction.dat`, `scattering.dat`, `wav.dat`)	0.1–$3\times10^{4}\,\mu$m = 4.1×10^{-5}–**12.4 eV**	absorption and scattering $\tau/N_{\rm H}$ only. **No** $\langle\cos\theta\rangle$ **product exists anywhere in the release.**
Zubko ZDA optics tables	10^{-3}–$10^{4}\,\mu$m = 1.24×10^{-4} eV–**1.24 keV**	$Q_{\rm abs}$, $Q_{\rm sca}$, $Q_{\rm ext}$, g for each radius

Table 5: What the model authors published, and how far it reaches.

The consequence is blunt: **the astrodust EUV band has no external reference at all**, because the HD23 release stops at 12.4 eV — exactly where the extension begins. Its accuracy claim rests entirely on the internal argument of §7.2 and on the seam being continuous.

The DL07 model *can* be checked, because Draine published the same model over the whole range. Comparing `data/kext_dl07_MW_euv.dat` with `kext_albedo_WD_MW_3.1_60_D03.all_2003` on the reference's own wavelengths (and counting only points where the reference grid is no finer than ours — elsewhere the comparison measures

our grid, not our optics) gives, per decade in λ, mean $|\Delta C_{\rm ext}|/C_{\rm ext}$ of 0.056–0.86%, mean $|\Delta\,\mathrm{albedo}| \leq 0.0019$ and mean $|\Delta\langle\cos\theta\rangle| \leq 0.00054$. `calc_kext.x dl07 euv` prints that table. The two residual features are worth naming: the excluded points cluster at the X-ray absorption edges (Fe K, Si K, Mg K, Fe L, O K, C K), which the reference brackets far more finely than our $\Delta\ln\lambda = 0.0116$ log grid can resolve; and the largest single deviation, 7.9% in the 1–10 μm decade, is confined to the 6.2, 7.6 and 7.9 μm PAH C–C features, where Draine's 2003 table uses a different vintage of the PAH cross sections.

8. Standalone programs

`make` in `SEDust/sed/` builds five executables. All of them are run *from* `SEDust/sed/`, because every data path is relative to `../`.

8.1 `calc_kext.x` — size-integrated transport optics

Builds a model and calls `size_integrated_extinction` on it, then writes the result under `../data/`. Every model goes through the same two calls, so the files it writes *are* the optics an RT host gets in memory — these are the files `dust_extinction` reads back (§5.).

```
./calc_kext.x astrodust [euv | <lam_min in um>]
./calc_kext.x dl07 [euv | <lam_min in um>]
./calc_kext.x zubko [formula | table]
./calc_kext.x from_files <descriptor> [data_dir]
```

With no argument it prints its usage and stops. `euv` carries the grid down to whatever the model's own dielectric function covers; an explicit number requests a specific floor in μm. `zubko` defaults to `formula`. `from_files` takes the descriptor path, and the data directory defaults to the descriptor's own directory.
Columns are

```
lambda[um]  albedo  <cos>  C_ext/H  C_abs/H  C_sca/H
```

in μm and cm^2 per H nucleon, followed by $K_{\rm abs}$ [cm^2/g] and — for `kext_astrodust_MW.dat` only — an eighth column $C_{\rm polext}$/H (Table 3). The first four columns are in the order of Draine's `kext_albedo` tables, so a reader written for those files (for instance the `par%ion_dust_kext` reader of the MoCHII host, which takes four reals from each row and uses λ, albedo and $C_{\rm ext}$) parses these unchanged. Note the fifth column differs from Draine's: his is $K_{\rm abs}$ [cm^2/g], ours is $C_{\rm abs}$/H [cm^2/H], so no dust-mass normalization is folded into the transport columns.

The $K_{\rm abs}$ column. Every product carries it, for every model. It is $C_{\rm abs}/\mathrm{H}$ divided by the model's dust mass per H, Eq. (2), which the model reports through `dust_mass_per_H` — so the column and the library agree by construction rather than by a constant copied between them. The normalization is one wavelength-independent number, as well defined in the extreme ultraviolet as in the optical, and the header of each file states its value and where that model's grain densities came from. The one product that would not get the column is a file-defined model whose optics tables declare no density; then $M_{\rm dust}/N_{\rm H} = 0$, the program says so, and writes the six transport columns alone.

Why the EUV products carry no polarized column. Of the four products, only `kext_astrodust_MW.dat` — the unextended one — has the dichroic column. Below 0.0912 μm that quantity is the announced zero of §7.3, and writing it into a data file would turn a documented deficit into a number a host cannot tell from a measurement. The EUV products are therefore scalar transport optics only, and are byte-for-byte the same files the scalar branch (v1.00) writes — which is also the cleanest demonstration that the scalar path is identical in the two branches.
After writing, the program checks $C_{\rm ext} = C_{\rm abs} + C_{\rm sca}$ and the ranges of albedo and $\langle\cos\theta\rangle$, and — for astrodust and DL07 — prints the comparison against the author-provided data of Table 5.

These files are transport optics, not a dust model. They carry no size-resolved $C_{\rm abs}(\lambda, a)$, no grain enthalpy and no Planck integral, so stochastic heating and thermal emission cannot be computed from them. A host that needs emission must build the model and take the same curves from `dust_extinction`, which is what keeps the absorbed power and the re-emitted spectrum referring to one and the same grain.

8.2 `main_astrodust.x` — the astrodust production SED

Solves the astrodust+PAH SED for a single Mathis-ISRF cell at $U = 1.585$ ($\log U = 0.20$, the HD23 best fit), on a 200-point log T grid from 2.7 to 5000 K. Reads the T-matrix Q table `../tmatrix/output/q_astrodust_P0.20_Fe0.00_1.400.dat` and `../data/release/size_distribution.dat`; writes `output/astrodust_irem_ours_<tag>S1.dat`, `. . . S2.dat` and `. . . PAH.dat`, each with λ [μm] and $\lambda I_\lambda / N_{\rm H}$. Optional arguments, in any order, each of which tags the output filename so a non-default run cannot clobber the production one: `qm`, `qm_dbcon`, `draine` (solver); `logU=X`; `nstate=N`, `nisrf=N` (QM solver sizes); `induced` (turn the stimulated-emission factor on); `photcut`; `c2` (Stage-1 density-corrected prefactor); `mathis_orig`.

8.3 `main_dl07.x` — the DL07 reference SED

Solves the Draine & Li (2007) silicate + carbonaceous SED at $U = 1$ with the WD01 size distribution. `./main_dl07.x [model] [qm|draine]`, where `model` is one of `mw31_00 . . . mw31_60` (default `mw31_60`), `lmc2_00`, `lmc2_05`, `lmc2_10`, `smc`; an unknown name is rejected with the valid list. Writes `output/dl07_sed_ours_<model>[_qm|_draine].dat` with columns λ, total, silicate and carbonaceous $\lambda I_\lambda / N_{\rm H}$.

8.4 `main_zubko.x` — the Zubko/ZDA SED, and the wide-grid regression

Solves the Zubko, Dwek & Arendt (2004) BARE-GR-S SED — PAH + graphite + silicate, the ZDA size-distribution formula, the DustEM optics and calorimetry tables — for a Mathis ISRF at $U = 1$, and writes `output/zubko_sed_ours[_<solver>][_euv].dat` with columns λ, total, PAH, graphite and silicate $\lambda I_\lambda / N_{\rm H}$.

```
./main_zubko.x [heuristic | draine | qm | equil] [euv]
```

The solver defaults to `heuristic`, the model default. `euv` replaces the Mathis field below the Lyman limit by a diluted 10^5 K blackbody, so that the field really does carry photons above 13.6 eV; the dilution is chosen so that band deposits

roughly as much power as the whole Mathis field — enough for the hard photons to matter, little enough that the small grains still heat stochastically.

This is the emission counterpart of `./calc_kext.x zubko`, which exercises only the extinction size integral. It exists because Zubko is the one shipped model whose optics grid ($10^{-3}\,\mu$m) reaches far past the band a transported field occupies ($0.0912\,\mu$m for the Mathis ISRF), so anything in the stochastic-heating path that reads the grid where it should read the field shows up here and nowhere else. Running it with and without `euv` is the regression of §7.4.

8.5 `make_enthalpy.x` — enthalpy tables (diagnostic)

Tabulates $U(T, a_{\rm eff})$ for the two astrodust enthalpy stages and writes `output/enthalpy_S1.dat` and `output/enthalpy_S2.dat` (201 log-spaced temperatures from 2.7 to 5000 K $\times$ the 169 radii of `../data/dielectric/DH21_aeff`, T outer and a inner). It takes no arguments. **It is not required to compute an SED**: the solver calls the closed-form Debye sums directly in its inner loop. Use it to compare the two stages, to plot $U(T)$, or to spot-check against published DL01 values.

8.6 Polarized and diagnostic programs

Four more targets are built on request rather than by the default `make`: `calc_polext.x` (polarized extinction per H, checked against the HD23 release), and the three self-checking test programs `test_scalar_mode.x`, `test_scatmat_aligned.x` and `test_euv_extension.x`. The tests print `ALL CHECKS PASSED` or the failing check; build them explicitly, since `make` alone will not refresh a stale binary:

```
make test_scalar_mode.x test_scatmat_aligned.x test_euv_extension.x
```

8.7 Polarized optics at other wavelengths

These programs live in `SEDust/tmatrix/`, not `sed/`, and are run from there. They exist because **the library reads tables and never runs the T-matrix at run time** — not to keep the code small, but because convergence cannot be certified above $x \simeq 55$ (below), and a silently wrong value is worse than an absent one. A revision of the T-matrix core is planned, so read this as the current policy rather than a permanent one.

Polarized *transfer* at every wavelength is rarely wanted; the five optical bands that ship cover the usual case. What matters is that any other wavelength can be computed:

```
./run_q_jori.x lam L1 [L2 ...] [ja=JA1:JA2] [tag=NAME]
./run_q_jori.x lamfile PATH [ja=JA1:JA2] [tag=NAME] # one per line, '#' comments
./run_q_jori.x lammerge STEM FILE [FILE ...] # reassemble the ja= windows
```

Each writes a pair, `q_astrodust_jori_P0.20_Fe0.00_1.400.TAG.dat` and `.TAG.wave`, which is exactly what `qpol_euv_path` and `qpol_euv_wave_path` take. Because `build_Cpol` interpolates in $\log\lambda$, the pair needs **at least two wavelengths**.

`ja=JA1:JA2` splits the 169 radii across *processes*, not threads: Mishchenko's solver hands the converged T-matrix to `AMPL` through `COMMON /TMAT/` and keeps further working storage in COMMON blocks, two of them blank, so the core is not re-entrant. Measured on this machine: 9m22s of wall time for 34m48s of processor time, 57 processes over three wavelengths.

Convergence is certified, not assumed. In `lam`, `lamfile` and `euv` mode every node with $x \in (50, 60]$ is solved twice — (DDELT, NDGS) = $(10^{-3}, 2)$ and $(3 \times 10^{-3}, 3)$ — and rejected if the two disagree by more than 5%, in which case the geometric-optics limit is used instead. Above $x = 60$ that limit is used outright. This matters because **`IERR = 0` is not a sufficient condition for convergence above** $x \simeq 55$: at $\lambda = 0.0912\,\mu$m, $a = 0.9441\,\mu$m ($x = 65.04$) the solver returns `IERR = 0` together with a value about 50% away from its neighbors. At the seam $x = 51.7$ and 54.7 pass the cross-check while 57.9, 61.4, 65.0 and 68.9 fail. The older full-sweep and `range` modes keep the previous $x > 50$ rule, so they still reproduce the shipped table byte for byte.

What certification costs. Leaving $x > 60$ at the geometric-optics zero discards part of the dichroic extinction: nothing at the seam, $\sim 8\%$ of the band total at $0.031\,\mu$m, $\sim 20\%$ at $0.022\,\mu$m and $\sim 46\%$ at $0.0124\,\mu$m. The true value lies between the certified 1.06×10^{-4} and the $1/x$ extrapolation 3.14×10^{-4} — already about 1% of the 5.5×10^{-2} the V band reaches, so the residual is a small fraction of an already small number. $C_{\rm pol}$, which drives polarized *emission*, has no such gap: the geometric-optics limit obtains it from the opaque-grain Fresnel surface integral.

Scattering matrices are less flexible. `run_scatmat_aligned.x` already accepts wavelengths on the command line (`./run_scatmat_aligned.x 0.44 0.55 0.79`), but three things `run_q_jori.x` has are missing: its output stem is fixed, so a custom run **overwrites** the shipped five-band table; there is no merge mode, so bands cannot be *added* to an existing table (a full recomputation costs about 4.5 minutes per band); and it has neither a radius split nor the $x > 50$ cross-check above. Adding a band today therefore means regenerating the whole table and accepting the uncertified large-x treatment.

9. The model object and accessors

```
type :: dust_model_t
  character(len=32) :: name
  integer :: NA, NLAM, NT
  real(wp), allocatable :: lam(:) ! (NLAM) [um]
  real(wp), allocatable :: T_first(:) ! (NT) [K]
  type(grain_pop_t), allocatable :: pops(:)
  integer :: n_channel
  character(len=16), allocatable :: channel_name(:)
  logical :: use_induced_emission ! default .false.
  character(len=16) :: stoch_method ! default 'heuristic'
  logical :: verbose ! default .false.
```

```
end type
```

Diagnostics. `m%verbose` (default `.false.`) keeps the library solve path silent; set it `.true.` to print the same solver diagnostics as the CLI drivers.

Convenience accessors (all in `dust_lib`):

```
n = dust_nlam(m) ! number of wavelengths
nc = dust_n_channel(m) ! number of output channels
lam = dust_lambda(m) ! allocatable copy of the wavelength grid [um]
nm = dust_channel_name(m, ic) ! name of channel ic
md = dust_mass_per_H(m) ! dust mass per H nucleon [g/H]
```

`dust_mass_per_H` is the conversion from the library's per-H cross sections to a mass opacity, Eq. (2) and §5.. Each population carries the solid density of its material in `m%pops(ip)%rho_bulk` [g cm^{-3}]; a population left at 0 contributes nothing to the sum.

10. The generic file loader

`build_from_files` reads a small text *descriptor* and assembles a model from data files, with no code changes. One `pop:` line per population:

```
# descriptor.txt
name = mymodel
# pop: <grain_type> <channel> <optics> <dnda_table> <calorimetry> <rho(0=use file)>
pop: pah 1 PAH_28_1201_neu.dat SzDist_PAH.dat Graphitic_Calorimetry.dat 0.0
pop: gra 2 Gra_121_1201.dat SzDist_Gra.dat Graphitic_Calorimetry.dat 0.0
pop: sil 3 suvSil_121_1201.dat SzDist_Sil.dat Silicate_Calorimetry.dat 0.0
```

- `grain_type`: `'sil'`, `'pah'`, or `'gra'` (selects the heat-capacity/mode model used by the `'qm'` solver; ignored by the GD solvers, which use the supplied enthalpy directly).
- `channel`: integer output channel ($1\ldots$); populations sharing a channel are summed.
- `optics`: a DustEM/Zubko Q-table ($Q_{\rm abs}$ per radius and wavelength); $C_{\rm abs} = Q_{\rm abs}\pi a^2$.
- `dnda_table`: a 2-column table a [μm] and $\mathrm{d}n/\mathrm{d}a$ [cm^{-1} H^{-1}].
- `calorimetry`: a specific-enthalpy table $u(T)$ [erg/g]; $H(T,a) = u(T)\,\rho\,\frac{4\pi}{3}a^3$.
- `rho`: grain density [g cm^{-3}]; `0.0` uses the value in the optics-file header. The same number sets the population's `rho_bulk`, so it fixes both the enthalpy above and the model's dust mass per H, Eq. (2); an optics file that declares no density and a descriptor that supplies none leave the population out of that mass.

All files are sought under `data_dir`. The coded builders (`build_astrodust/dl07/zubko`) are the recommended path for the built-in models; `build_from_files` is for adding new data-defined models.

11. Linking into a 3D RT code

Compile your RT source against `libsedust.a` with the `.mod` search path pointing at `SEDust/sed/`:

```
gfortran -I<sed> my_rt.f90 <sed>/libsedust.a -fopenmp -o my_rt.x
```

Three complete examples are provided under `SEDust/sed/rt_example/`, each with its build command in its own header. No `ISO_C_BINDING` is needed — the RT code simply uses the Fortran module `dust_lib`.

- `use_dustlib.f90` — the scalar host flow, and the one to read first: build a model once (which is also where its extinction table is attached), take the transport optics from `dust_extinction` (including `gbar` and `albedo`), take one cell's emission from `dust_emission`, then build a second model with `lam_min` to show how a host that transports ionizing radiation widens the grid. It is byte-for-byte the same file in the scalar branch (v1.00).
- `use_dustlib_pol.f90` — where SEDust stops and the RT starts: `lamI_pol` and `Cpol_ext`, and the $\sin^2\gamma$, $F_{\rm turb}$ and position-angle factors the host must supply (§6.6).
- `use_dustlib_scatmat.f90` — the aligned-scattering query API along a two-cell photon path (§6.5).

Threading. `m` is read-only during `dust_emission`; all scratch is local. The solver uses OpenMP internally over grains. If your RT parallelizes over cells, take care to avoid oversubscription (nested parallelism); the default `'heuristic'` solver is serial per call, which suits an outer cell-level parallel loop.

12. Units and conventions

- Wavelengths `lam` are in μm throughout; cross sections in cm^2.
- `J_lam` is a mean intensity with the same units as the Planck function B_λ (specific intensity per steradian). The library's output $\lambda I_\lambda / N_{\rm H}$ is then in erg s^{-1} cm^{-2} sr^{-1} H^{-1}.
- The CMB (2.725 K) is included in the heating field by `J_Mathis`.
- The induced-/stimulated-emission factor is off by default (the output is *net* emission, matching the HD23 and DL07spec released SEDs).

13. Data files

Model data lives under `SEDust/data/`:

- astrodust / DL07: the T-matrix Q-table (`../tmatrix/output/q_astrodust_*.dat`) and the HD23 release size distribution (`../data/release/size_distribution.dat`).
- Zubko: `../data/zubko/` — the ZDA config/formula, the tabulated size distributions, the DustEM Q-tables, and the calorimetry tables from the Camps et al. 2015 benchmark; a ready descriptor is `../data/zubko/zubko_descriptor.txt`.

- Polarized optics: `../data/dielectric/q_DH21Ad_*.dat.gz` (the HD23 release orientation-resolved Q table, the default) and `../tmatrix/output/q_astrodust_jori_*.dat.gz` (SEDust's own regenerated one, opt-in through `qpol_path`, and the only one with a birefringence block); plus `../tmatrix/output/scatmat_aligned_*.dat.gz` (aligned scattering matrix), with their grid axes `../data/dielectric/DH21_wave` and `DH21_aeff`. The EUV companion table `q_astrodust_jori_euv_*.dat.gz` does *not* ship (§7.3); `DH21_wave_euv` is the axis it would be computed on.
- Size-integrated transport optics, written by `calc_kext.x`, see §8. — `../data/kext_astrodust_MW.dat` (1129 rows, 0.0912–39810 μm, with the dichroic 8th column), `kext_astrodust_MW_euv.dat` (1719 rows, from 1.001×10^{-4} μm), `kext_dl07_MW.dat` (1129 rows, 0.0912–39810 μm), `kext_dl07_MW_euv.dat` (1761 rows, from 6.205×10^{-5} μm) and `kext_zubko_BARE_GR_S.dat` (1201 rows, 10^{-3}–10^{4} μm). A host that reads a file reads one of these; a host that links the library gets the same curves from `dust_extinction`, which reads one of these files too — the two EUV products and the Zubko one are the defaults, see §5.. The DL07 and astrodust unextended products carry exactly the long-wavelength rows of their EUV counterparts, bit for bit.